```
        x = torch.relu(x)
        x = self.conv4(x)
        x = torch.relu(x)

        _,_,cH,cW = x.shape
        xcoords = torch.arange(cW).repeat(cH,1).float() / cW
1
        ycoords =
torch.arange(cH).repeat(cW,1).transpose(1,0).float() / cH
        spatial_coords = torch.stack([xcoords,ycoords],dim=0)
        spatial_coords = spatial_coords.unsqueeze(dim=0)
        spatial_coords = spatial_coords.repeat(N,1,1,1)
        x = torch.cat([x,spatial_coords],dim=1)
        x = x.permute(0,2,3,1)
        x = x.flatten(1,2)

        K = self.k_proj(x)
2
        K = self.k_norm(K)

        Q = self.q_proj(x)
        Q = self.q_norm(Q)

        V = self.v_proj(x)
        V = self.v_norm(V)
        A = torch.einsum('bfe,bge->bfg',Q,K)
3
        A = A / np.sqrt(self.node_size)
        A = torch.nn.functional.softmax(A,dim=2)
        with torch.no_grad():
            self.att_map = A.clone()
        E = torch.einsum('bfc,bcd->bfd',A,V)
4
        E = self.linear1(E)
        E = torch.relu(E)
        E = self.norm1(E)
        E = E.max(dim=1)[0]
        y = self.linear2(E)
        y = torch.nn.functional.log_softmax(y,dim=1)
        return y
```

- ***1*** Appends the (x,y) coordinates of each node to its feature vector and normalizes to within the interval [0, 1]
- ***2*** Projects the input node matrix into key, query, and value matrices
- ***3*** Batch matrix multiplies the query and key matrices
- ***4*** Batch matrix multiplies the attention weight matrix and the value matrix